

CSE 152A Winter 2025 – Assignment 1

- Assignment Published On: **Friday, Jan 17, 2025**
- Due On: **Mon, Jan 27, 2025 11:59 PM (Pacific Time)**

Instructions

Please answer the questions below using Python in the attached Jupyter notebook and follow the guidelines below:

- This assignment must be completed **individually**. For more details, please follow the Academic Integrity Policy and Collaboration Policy posted on lecture slides.
- All the solutions must be written in this Jupyter notebook.
- After finishing the assignment in the notebook, please export the notebook as a PDF and submit both the notebook and the PDF (i.e. the `.ipynb` and the `.pdf` files) on Gradescope. (Note: Please ensure that all images/plots are clear in the pdf).
- You may use basic algebra packages (e.g. `NumPy`, `SciPy`, etc) but you are not allowed to use open source codes that directly solve the problems. Feel free to ask the instructor and the teaching assistants if you are unsure about the packages to use.
- It is highly recommended that you begin working on this assignment early.
- Make sure that you read hints for questions (wherever given).

Late Policy: Assignments submitted late will receive a 25% grade reduction for each 12 hours late (that is, 50% per day).

Submission Instructions

- You must submit both the `.ipynb` file and a `.pdf` version of your notebook.
- Some methods to generate PDF
 - File -> Save and Export Notebook As -> `.html` -> Print -> Save as PDF
 - Using [nbconvert](#)
- We will be grading primarily from your notebook.
 - It is your responsibility to make sure that your code and outputs are visible.

Virtual Environment

Initial Set-up

You can utilize a virtual environment (`venv`) in order to manage dependencies: [venv link](#) along with the libraries specified in `requirements.txt` .

To create the environment:

```
python -m venv cse152a_hw1
```

To activate the environment:

```
source ./cse152a_hw1/bin/activate
```

Once the virtual environment is activated, you can install the libraries according to `requirements.txt` like so:

```
pip install -r requirements.txt
```

You should then add the environment to jupyter notebook like so:

```
python -m ipykernel install --user --name=cse152a_hw1
```

To deactivate the environment, simply:

```
deactivate
```

This assumes you have Python installed already.

Using the venv

Once you've installed all the requirements within the venv, you can deactivate and would no longer need to repeat the above steps. You can simply open a Jupyter instance.

Opening a Jupyter instance:

```
jupyter-lab
```

Ensure that you select the correct kernel (named `cse152a_hw1` if you followed the steps above) by clicking Kernel -> Change Kernel.

```
In [1]: # Run this cell to check that all libraries are installed  
import numpy as np  
import matplotlib.pyplot as plt  
import pickle  
from scipy.signal import convolve
```

```
%matplotlib inline
from mpl_toolkits.mplot3d import Axes3D
from scipy.ndimage import zoom
```

Problem 1: Photometric Stereo [20 pts]

The goal of this problem is to implement Lambertian photometric stereo.

Note that the albedo is unknown and non-constant in the images you will use.

As input, your program should take in multiple images along with the light source direction for each image.

Data

You will use synthetic images as data. These images are stored in `.pickle` files which were graciously provided by Satya Mallick. Each `.pickle` file contains

- `im1`, `im2`, `im3`, `im4`, ... images.
- `l1`, `l2`, `l3`, `l4`, ... light source directions.

You will find all the data for this part in `synthetic_data.pickle`.

Note: Make sure to use `%matplotlib inline` to enable plotting by matplotlib in jupyter notebook/lab IDE

```
In [2]: ## Example: How to read and access data from a pickle
pickle_in = open("data.pickle", "rb")
data = pickle.load(pickle_in, encoding="latin1")

# data is a dict which stores each element as a key-value pair.
print("Keys: " + str(data.keys()))

# To access the value of an entity, refer it by its key.
print("Image:")
plt.imshow(data["im1"], cmap = "gray", vmin=0.0, vmax=1.0)
plt.show()

print("Light source direction: " + str(data["l1"]))

plt.imshow(data["im2"], cmap = "gray", vmin=0.0, vmax=1.0)
plt.show()

print("Light source direction: " + str(data["l2"]))

plt.imshow(data["im3"], cmap = "gray", vmin=0.0, vmax=1.0)
plt.show()

print("Light source direction: " + str(data["l3"]))

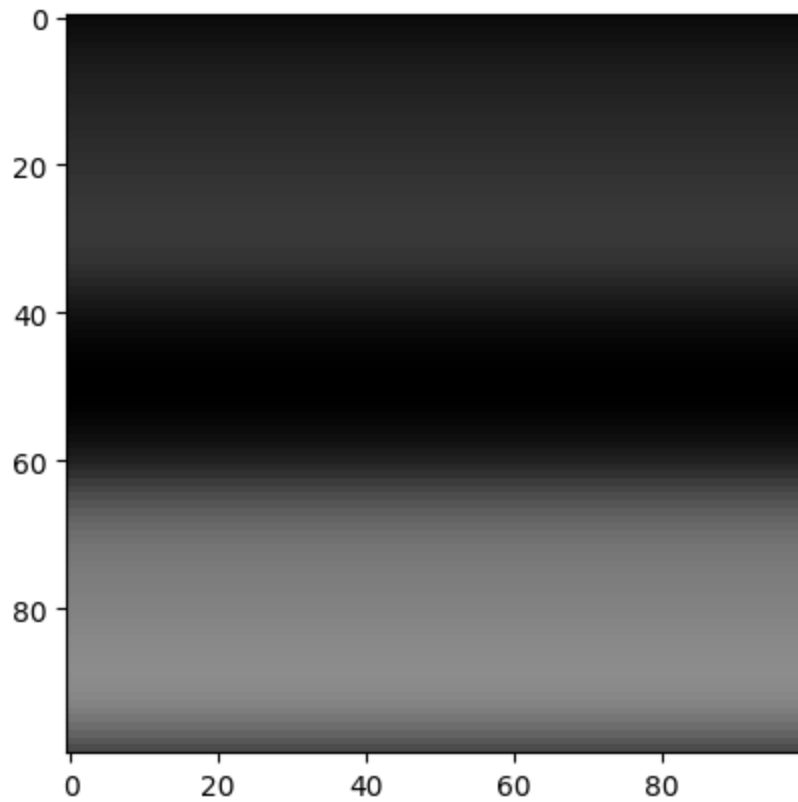
plt.imshow(data["im4"], cmap = "gray", vmin=0.0, vmax=1.0)
```

```
plt.show()

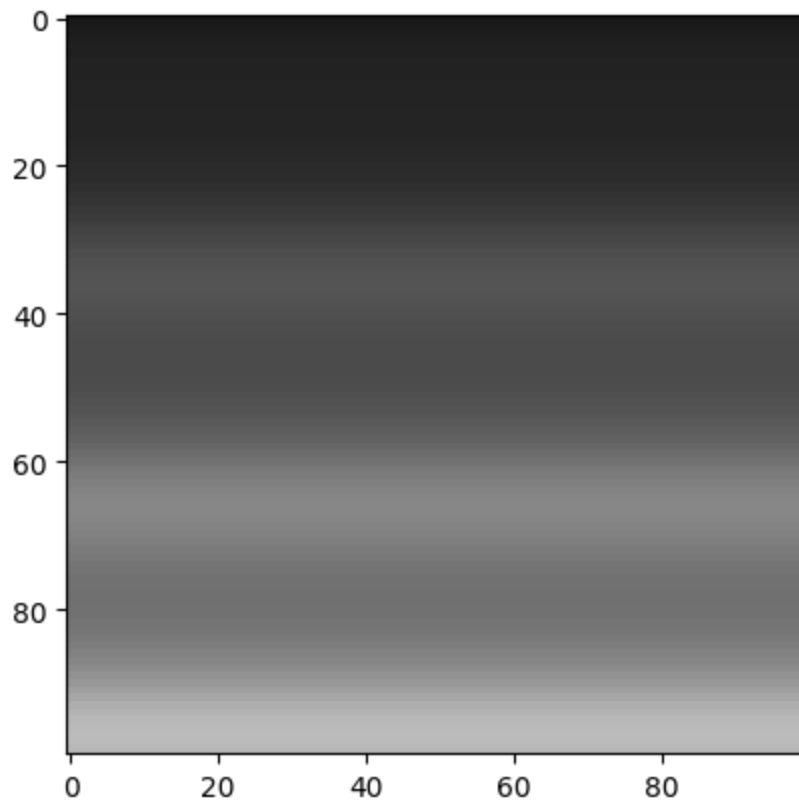
print("Light source direction: " + str(data["l4"]))
print(f"Image shape: {data['im1'].shape}")
```

Keys: dict_keys(['im1', 'im2', 'im3', 'im4', 'l1', 'l2', 'l3', 'l4'])

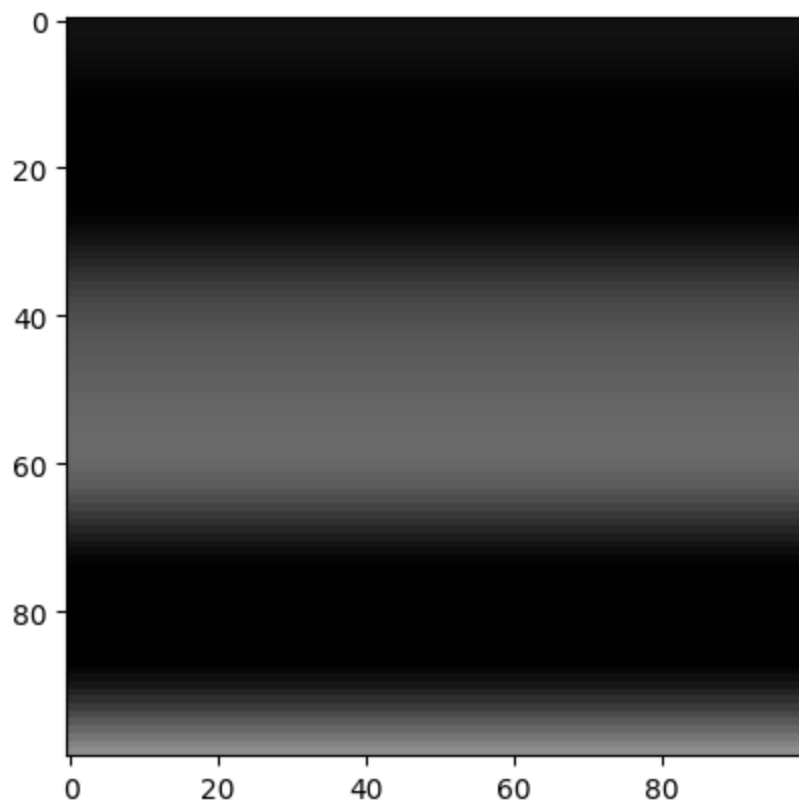
Image:



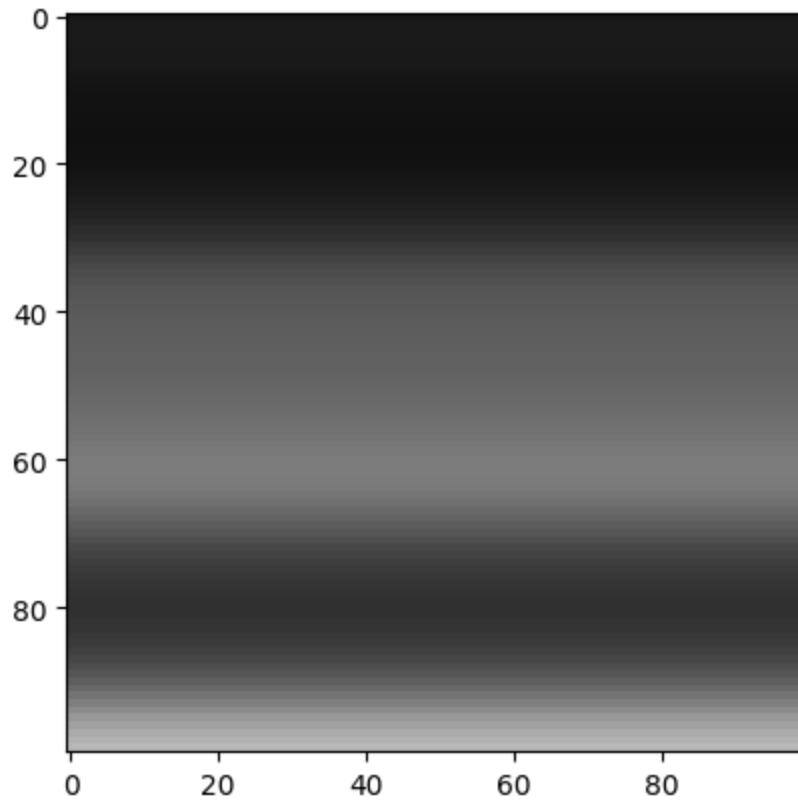
Light source direction: [0.57735027 0.57735027 0.57735027]



Light source direction: $[0. \ 0. \ 1.]$



Light source direction: $[-0.42426407 \ -0.70710678 \ 0.56568542]$



Light source direction: [0.21821789 -0.43643578 0.87287156]
 Image shape: (100, 100)

1(a) Photometric Stereo [8 pts]

Implement the photometric stereo technique described in the lecture. Your program should have two parts:

- Read in the images and corresponding light source directions, and estimate the surface normals and albedo map.
- Reconstruct the depth map from the surface with the implementation of the Horn integration technique given below in `horn_integrate` function. Note that you will typically want to run the `horn_integrate` function with 10000 - 100000 iterations, meaning it will take a while.

Note:

- You MUST solve the system of equations as discussed in lecture
 - You may NOT use functions to solve it for you such as `np.linalg.pinv()`, `np.linalg.solve()`, `np.linalg.lstsq()`.
- The following `horn_integrate()` function is not to be edited for the purpose of solving this homework. Consider this to be a blackbox!

```
In [3]: def horn_integrate(gx, gy, niter=25000):
        """
        horn_integrate recovers the function g from its partial
        derivatives gx and gy.
```

mask is a binary image which tells which pixels are involved in integration.
 niter is the number of iterations.
 typically 100,000 or 200,000,
 although the trend can be seen even after 1000 iterations

Parameters:

gx: 2D array (H,W)
 gy: 2D array (H,W)
 mask: 2D array (H,W)
 niter: int

Return: g: 2D array (H,W)
 """

```
g = np.ones(np.shape(gx))
mask = np.ones_like(g)

gx = np.multiply(gx, mask)
gy = np.multiply(gy, mask)

A = np.array([[0,1,0],[0,0,0],[0,0,0]]) #y-1
B = np.array([[0,0,0],[1,0,0],[0,0,0]]) #x-1
C = np.array([[0,0,0],[0,0,1],[0,0,0]]) #x+1
D = np.array([[0,0,0],[0,0,0],[0,1,0]]) #y+1

d_mask = A + B + C + D

den = np.multiply(convolve(mask,d_mask,mode="same"),mask)
den[den == 0] = 1
rden = 1.0 / den
mask2 = np.multiply(rden, mask)

m_a = convolve(mask, A, mode="same")
m_b = convolve(mask, B, mode="same")
m_c = convolve(mask, C, mode="same")
m_d = convolve(mask, D, mode="same")

term_right = np.multiply(m_c, gx) + np.multiply(m_d, gy)
t_a = -1.0 * convolve(gx, B, mode="same")
t_b = -1.0 * convolve(gy, A, mode="same")
term_right = term_right + t_a + t_b
term_right = np.multiply(mask2, term_right)

for k in range(niter):
    g = np.multiply(mask2, convolve(g, d_mask, mode="same")) + term_right

return g
```

In [4]: **def** photometric_stereo(images, lights):

"""

Implement the photometric stereo equations as described in lecture.

Parameters:

images: 3D array (n_imgs, h, w)
 lights: 2D array (n_imgs, 3)

```

Return: tuple of (albedo, normals, H_horn)
    albedo: 2D array (h,w)
    normals: 3D array (h,w,3)
    H_horn: 2D array (h,w)
"""

albedo = np.ones(images[0].shape)
normals = np.dstack((np.zeros(images[0].shape),
                     np.zeros(images[0].shape),
                     np.ones(images[0].shape)))
H_horn = np.ones(images[0].shape)

""" YOUR CODE HERE """
'''Square = lights.T @ lights
Sprime = np.linalg.inv(Square)
b = Sprime @ lights.T @ images'''
Square = np.dot(lights.T, lights)
#Ims = np.dot(lights.T, images)
Inv = np.linalg.inv(Square)
height, width = images.shape[1], images.shape[2]
for i in range(height):
    for j in range(width):
        I = images[:, i, j] #per pix :(
        b = np.dot(Inv, np.dot(lights.T, I))

        albedo[i, j] = np.linalg.norm(b)
        if albedo[i, j] > 0: # Avoid division by zero
            normals[i, j] = b / albedo[i, j]
#albedo = np.linalg.norm(b)
#normals = b/albedo
gx = -normals[:, :, 0]/normals[:, :, 2]
gy = -normals[:, :, 1]/normals[:, :, 2]
H_horn = horn_integrate(gx, gy)

return albedo, normals, H_horn

```

1(b) Display outputs using `im1`, `im2` and `im4` [6 Points]

- The estimated albedo map.
- The estimated surface normals by showing both
 - Needle map
 - Three images showing the components of the surface normal
- A wireframe of the depth map

An example of the outputs are shown below. Your results will not match these. We've provided code below to help you visualize them.

 Problem1%20example.png

```

In [5]: def visualize(albedo, normals, horn_depth):
        """
        The following code is used to generate the albedo plot,
        the three normal component plots, and the wireframe
        depth map plot.

        Parameters:
            albedo: 2D array (h, w)
            normals: 3D array (h, w, 3)
            horn_depth: 2D array (h,w)
        """

        # Stride in the plot, you may want to adjust it to different images
        stride = 8

        # showing albedo map
        fig = plt.figure()
        albedo_max = albedo.max()
        albedo = albedo / albedo_max
        plt.imshow(albedo, cmap="gray", vmin=0.0, vmax=1.0)
        plt.show()

        # showing normals as three separate channels
        figure = plt.figure()
        ax1 = figure.add_subplot(131)
        ax1.imshow(normals[..., 0], vmin=0.0, vmax=1.0)
        ax2 = figure.add_subplot(132)
        ax2.imshow(normals[..., 1], vmin=0.0, vmax=1.0)
        ax3 = figure.add_subplot(133)
        ax3.imshow(normals[..., 2], vmin=0.0, vmax=1.0)
        plt.show()

        # showing normals as quiver
        X, Y, _ = np.meshgrid(np.arange(0, np.shape(normals)[0], stride),
                               np.arange(0, np.shape(normals)[1], stride),
                               np.arange(1))

        X = X[..., 0]
        Y = Y[..., 0]
        Z = horn_depth[::stride, ::stride]
        NX = normals[..., 0][::stride, ::stride]
        NY = normals[..., 1][::stride, ::stride]
        NZ = normals[..., 2][::stride, ::stride]
        fig = plt.figure(figsize=(5, 5))
        ax = fig.add_subplot(projection='3d')
        plt.quiver(X, Y, Z, NX, NY, NZ, length=2, antialiased=True)
        #ax.view_init(elev=30, azim=340)# -> Can uncomment this if you want to adjust t
        ax.set_xlabel("X")
        ax.set_ylabel("Y")
        ax.set_zlabel("Z")
        plt.show()

        # plotting wireframe depth map

        H = horn_depth[::stride, ::stride]
        fig = plt.figure()

```

```

ax = fig.add_subplot(projection='3d')
ax.plot_surface(X,Y,H)
# ax.view_init(elev=30, azim=340)
ax.set_xlabel("X")
ax.set_ylabel("Y")
ax.set_zlabel("Z")
plt.show()

```

```

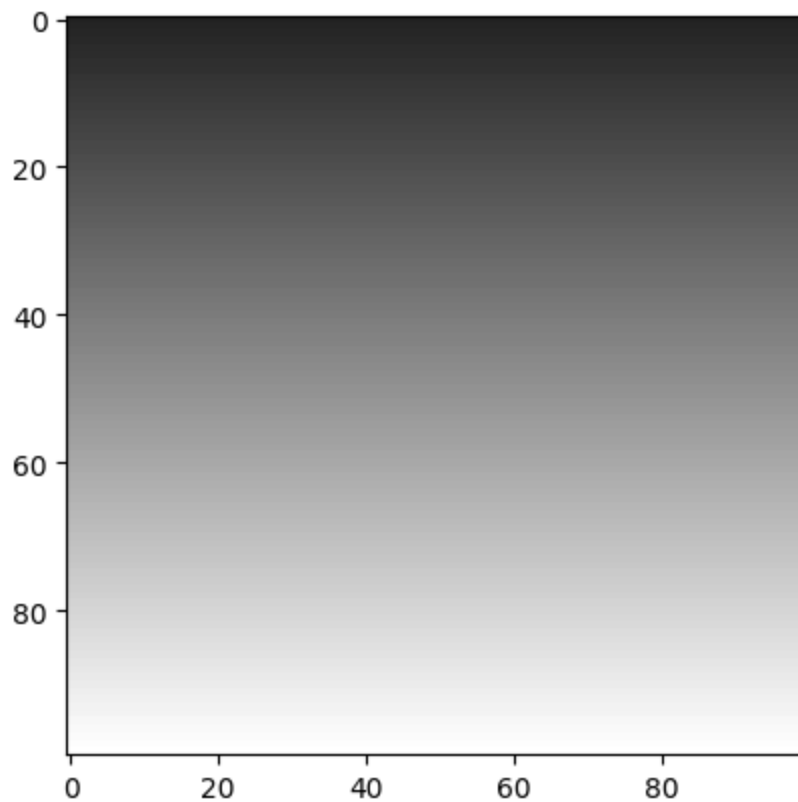
In [6]: # Visualize the results using the three images-light source pairs:
# (im1, l1), (im2, l2), and (im4, l4)
lights = np.vstack((data["l1"], data["l2"], data["l4"]))

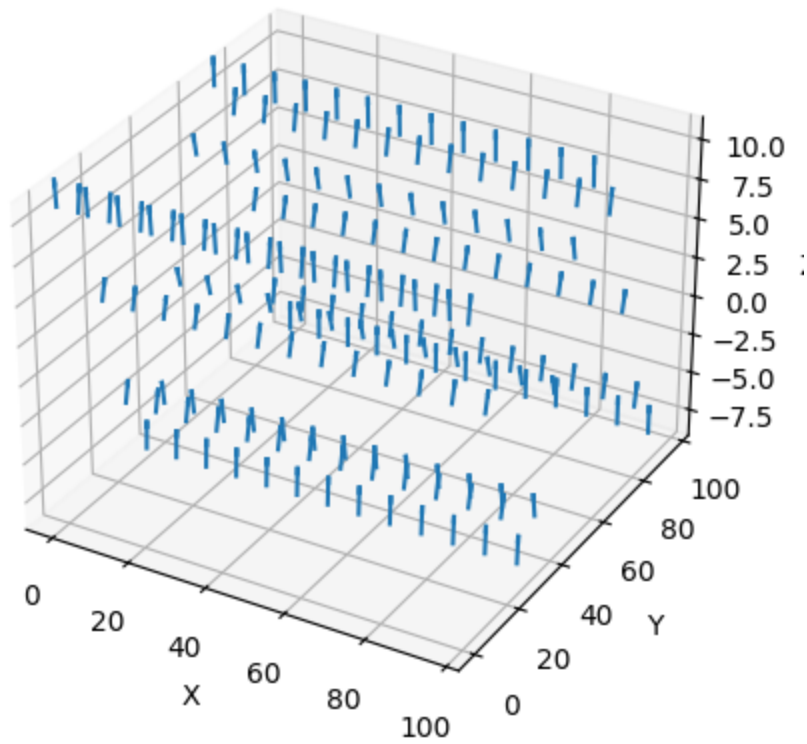
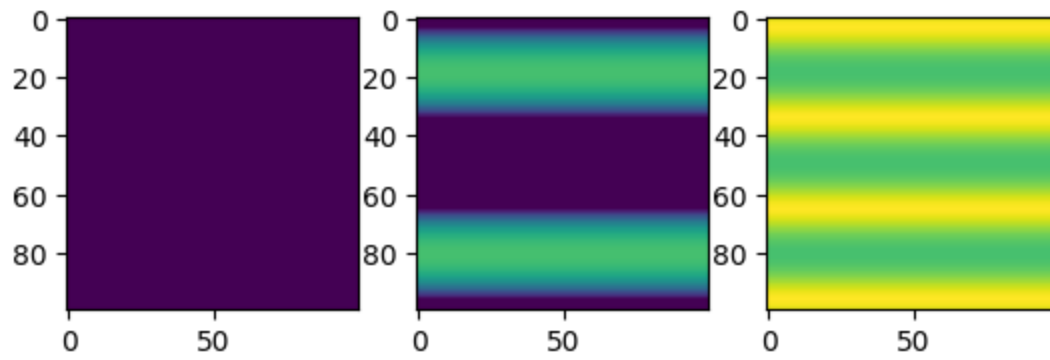
images = []
images.append(data["im1"])
images.append(data["im2"])
images.append(data["im4"])
images = np.array(images)

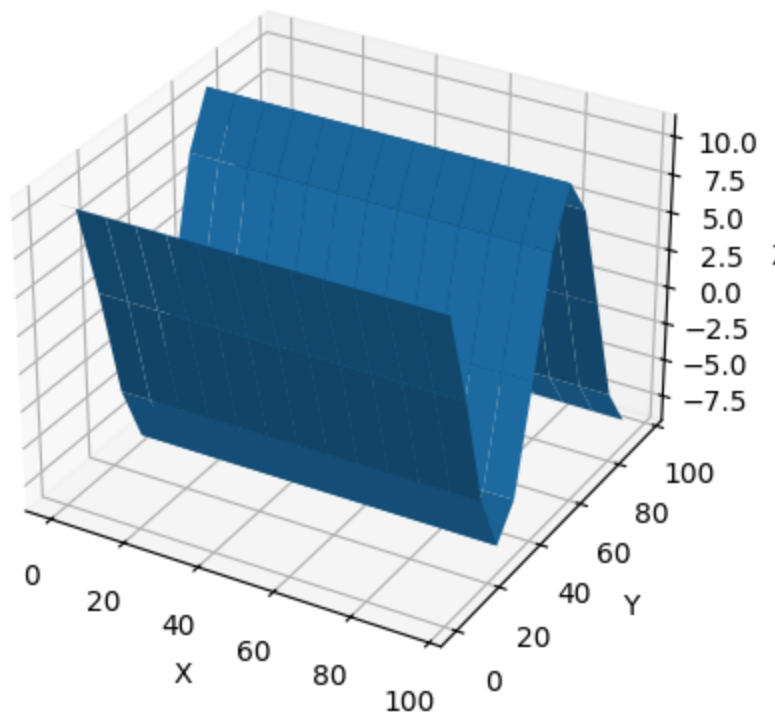
albedo, normals, horn_depth = photometric_stereo(images, lights)

visualize(albedo, normals, horn_depth)

```







1(c) Display outputs using all four images (most accurate result) [6 points]

- The estimated albedo map.
- The estimated surface normals by showing both
 - Needle map
 - Three images showing the components of the surface normal
- A wireframe of the depth map

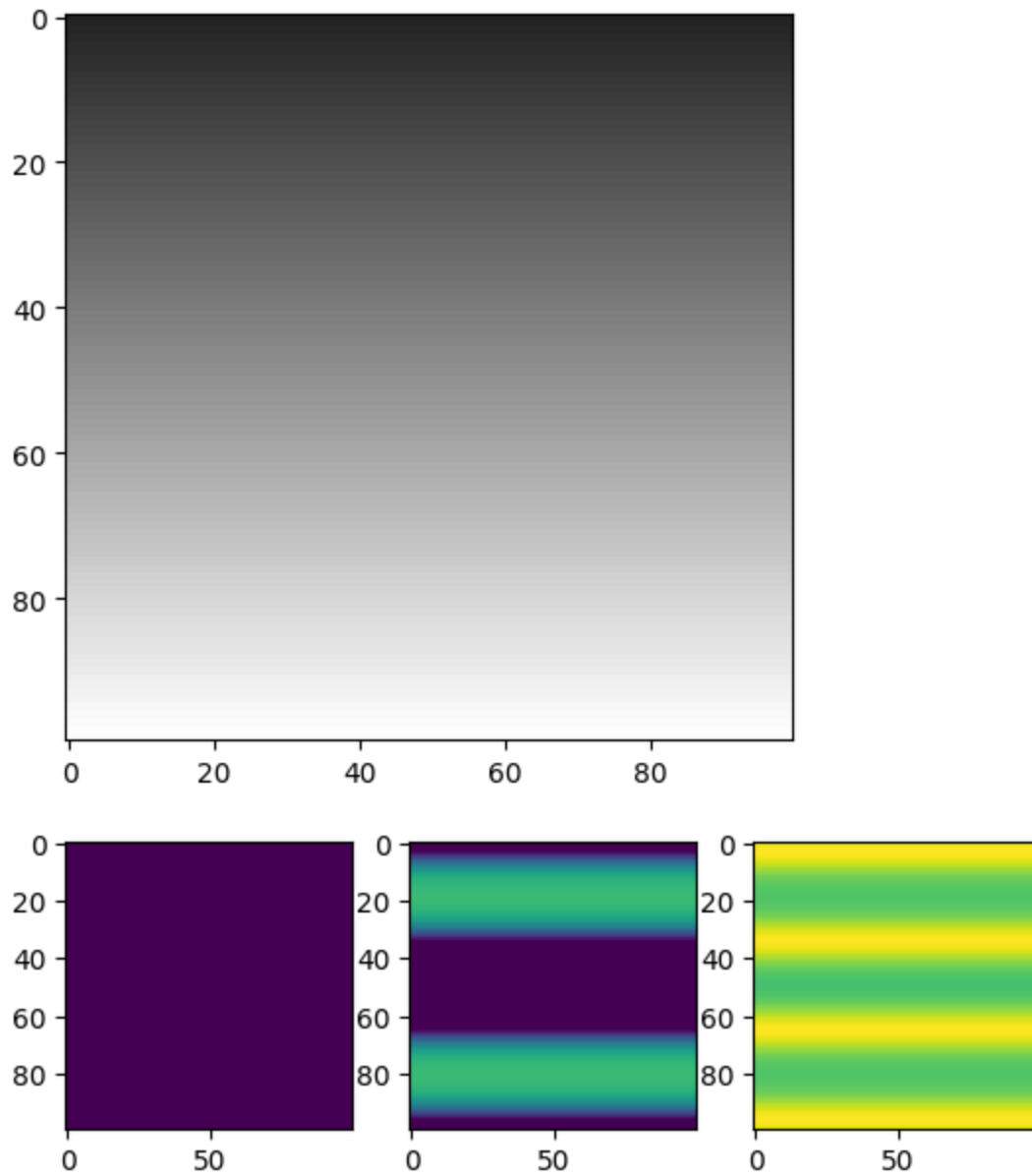
You can refer to the code used in the cell above and adapt it to display the outputs for all the four images i.e im1, im2, im3 and im4 instead of just the 3 images i.e im1, im2 and im4 used before.

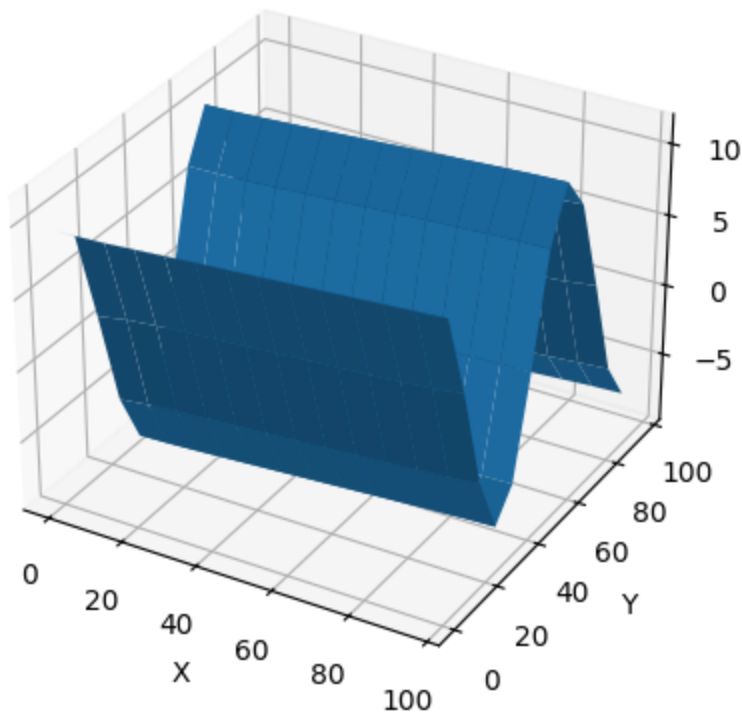
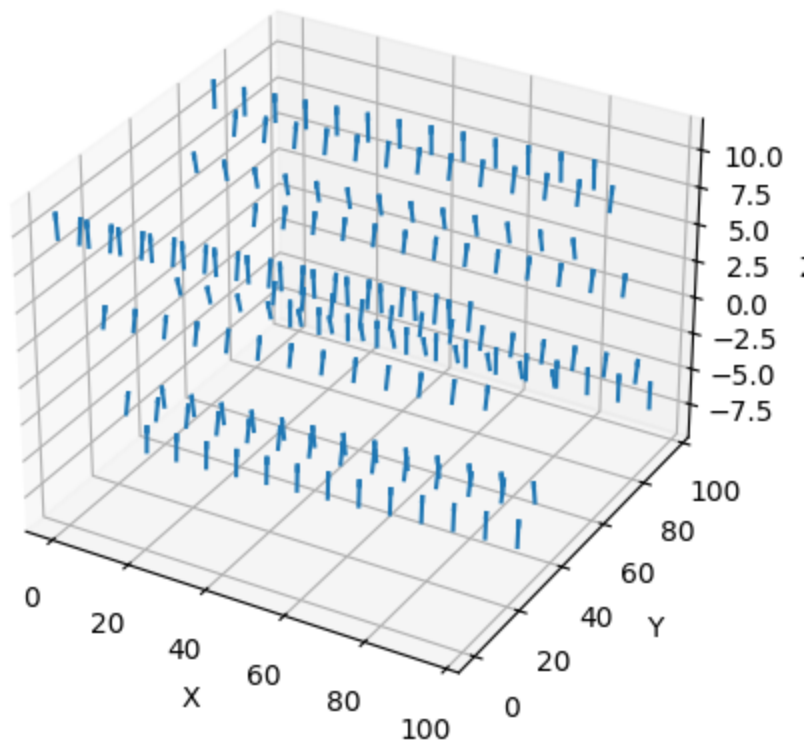
```
In [7]: # Visualize the results using the four images-light source pairs:
# (im1, l1), (im2, l2), (im3, l3) and (im4, l4)
""" YOUR CODE HERE """
lights = np.vstack((data["l1"], data["l2"], data["l3"], data["l4"]))

images = []
images.append(data["im1"])
images.append(data["im2"])
images.append(data["im3"])
images.append(data["im4"])
images = np.array(images)

albedo, normals, horn_depth = photometric_stereo(images, lights)
```

```
visualize(albedo, normals, horn_depth)
```





Problem 2: Image Rendering [20 points]

In this exercise, we will render the image of a face with two different point light sources using a Lambertian reflectance model. We will use two albedo maps, one uniform and one that is more realistic. The face heightmap, the light sources, and the two albedo are given in

`facedata.npy` for Python (each row of the `lightsource` variable encode a light location). The data from `facedata.npy` is already provided to you.

Note: Please make good use out of subplot to display related image next to each other.

2(a) Plot the face in 2-D [2 pts]

Plot both albedo maps using `imshow`. Explain what you observe in brief.

```
In [8]: # Load facedata.npy as ndarray
face_data = np.load('facedata.npy', encoding='latin1', allow_pickle=True)

# Load albedo matrix
albedo = face_data.item().get('albedo')

# Load uniform albedo matrix
uniform_albedo = face_data.item().get('uniform_albedo')

# Load heightmap
heightmap = face_data.item().get('heightmap')

# Load light source
light_source = face_data.item().get('light_source')
```

You should use the following link to learn about subplots:

https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.subplots.html

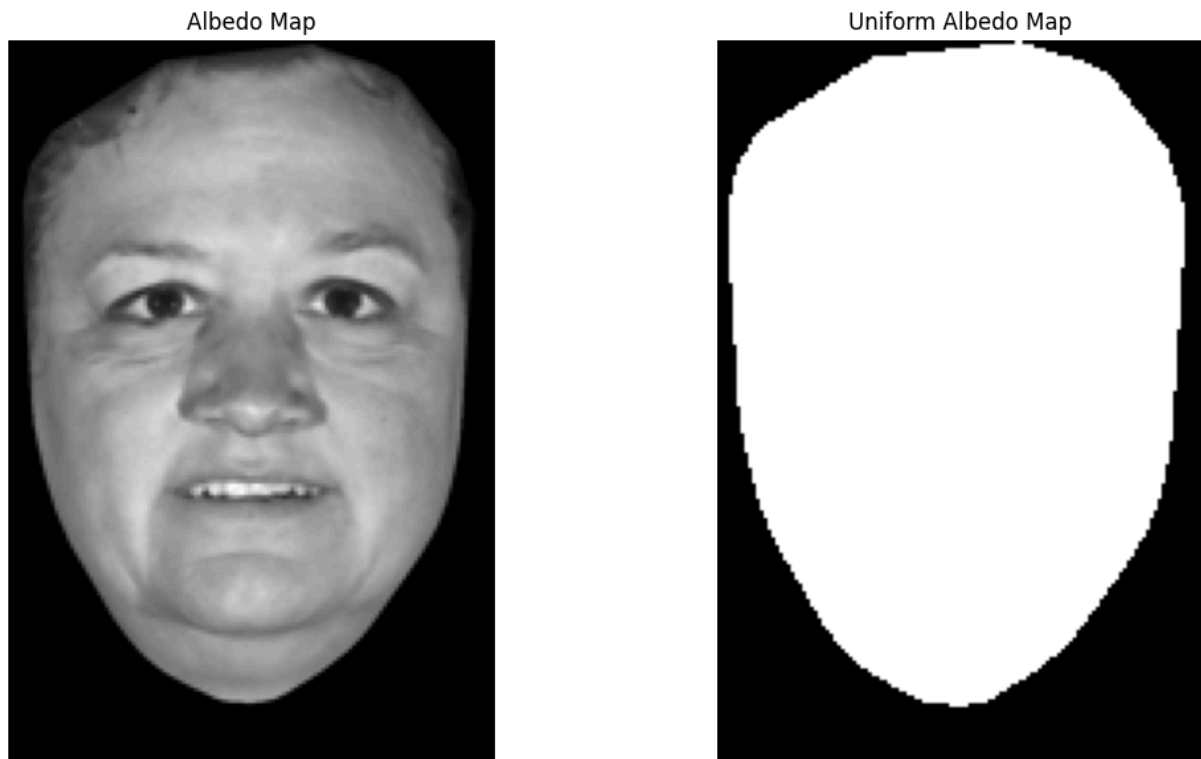
Make sure to use `cmap=gray` while showing the images in grayscale format otherwise you can expect to see green-yellow colors.

```
In [9]: # Plot the face in 2-D (plot both albedo maps using imshow)
        """ YOUR CODE HERE """
plt.figure(figsize=(12, 6))

# Albedo map
plt.subplot(1, 2, 1)
plt.imshow(albedo, cmap="gray")
plt.title('Albedo Map')
plt.axis('off')

# Uniform albedo map
plt.subplot(1, 2, 2)
plt.imshow(uniform_albedo, cmap='gray')
plt.title('Uniform Albedo Map')
plt.axis('off')

plt.tight_layout()
plt.show()
```



Above are the albedo and uniform albedo *maps*. The albedo map has a full uniform reconstruction of a face using the found albedo matrix. The uniform one constructs an area where there is face and where there isn't. It would see the "uniform" property of this array has removed all features of the face.

2(b) Plot the face in 3-D [2 pts]

Using both the heightmap and the albedo, plot the face using `plot_surface`. Do this for both albedos.

Learn about creating mesh grids here:

<https://numpy.org/doc/stable/reference/generated/numpy.mgrid.html>

Alternatively, use <https://numpy.org/doc/stable/reference/generated/numpy.meshgrid.html>

Learn about plotting surfaces here:

[https://matplotlib.org/2.2.5/api/_as_gen/mpl_toolkits.mplot3d.axes3d.Axes3D.html?](https://matplotlib.org/2.2.5/api/_as_gen/mpl_toolkits.mplot3d.axes3d.Axes3D.html?highlight=plot_surface#mpl_toolkits.mplot3d.axes3d.Axes3D.plot_surface)

[highlight=plot_surface#mpl_toolkits.mplot3d.axes3d.Axes3D.plot_surface](https://matplotlib.org/2.2.5/api/_as_gen/mpl_toolkits.mplot3d.axes3d.Axes3D.plot_surface)

This example may help you to put things together:

https://matplotlib.org/2.0.2/examples/mplot3d/surface3d_demo.html

Hint: Make sure to set strides to one, cmap should be gray, linewidth is zero and antialiased is set to false. Figure out how to use albedo via the parameter `facecolors` -- you may want to look up `plt.cm.gray()` !

Currently, the height is stored along the **negative z-axis**. In the cell below, we've inverted it for visualization.

```
In [10]: heightmap.shape
```

```
Out[10]: (188, 126)
```

```
In [11]: albedo.shape
```

```
Out[11]: (188, 126)
```

```
In [12]: uniform_albedo.shape
```

```
Out[12]: (188, 126)
```

```
In [15]: # Plot the face in 3-D
# (Using the face_height & albedo plot the faces using plot_surface)
face_height = -heightmap # inverted heightmap for visualization

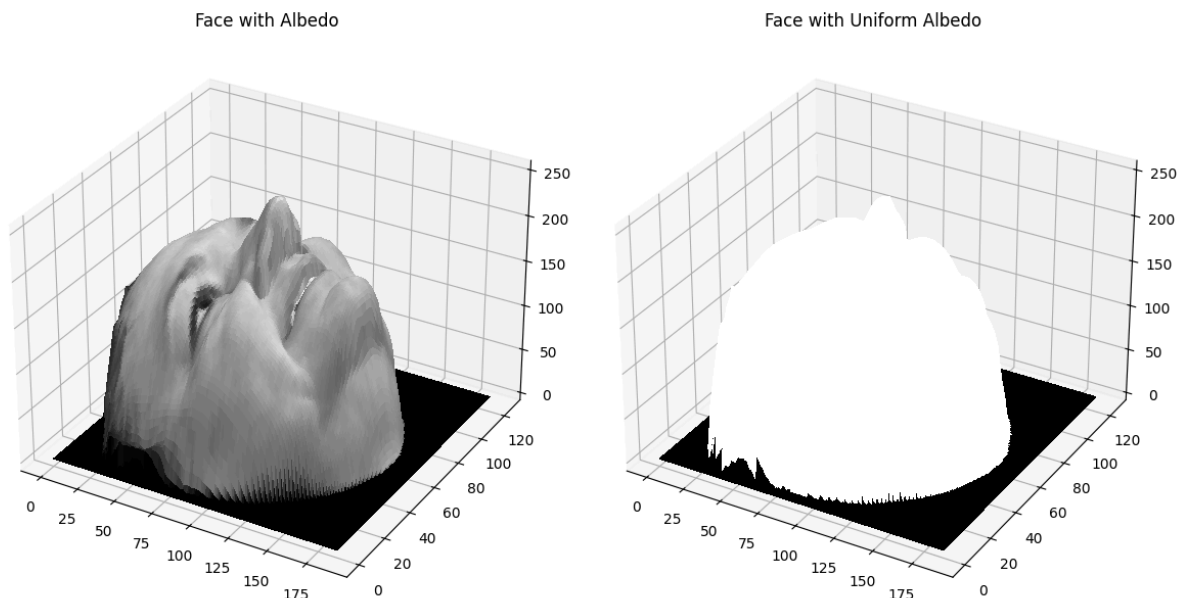
""" YOUR CODE HERE """
rows, cols = face_height.shape
X, Y = np.mgrid[0:rows, 0:cols]

fig = plt.figure(figsize=(12, 6))

#plot for 'albedo'
ax1 = fig.add_subplot(1, 2, 1, projection='3d')
surface1 = ax1.plot_surface(
    X, Y, face_height,
    rstride=1, cstride=1,
    facecolors=plt.cm.gray(albedo / albedo.max()), #normed for gray
    cmap='gray',
    linewidth=0,
    antialiased=False
)
ax1.set_title('Face with Albedo')

# plot for 'uniform albedo'
ax2 = fig.add_subplot(1, 2, 2, projection='3d')
surface2 = ax2.plot_surface(
    X, Y, face_height,
    rstride=1, cstride=1,
    facecolors=plt.cm.gray(uniform_albedo / uniform_albedo.max()),
    cmap='gray',
    linewidth=0,
    antialiased=False
)
ax2.set_title('Face with Uniform Albedo')

plt.tight_layout()
plt.show()
```



2(c) Surface normals [8 pts]

Calculate the surface normals and display them as a quiver plot using `quiver` in `matplotlib` in Python. Recall that the surface normals are given by

$$\left[-\frac{\delta f}{\delta x}, -\frac{\delta f}{\delta y}, 1\right]$$

Also, recall, that each normal vector should be normalized to unit length.

Note: You can use differencing to compute spatial derivatives of discrete functions. You can use `scipy`'s implementation on `convolve()` to achieve this. Learn about them here:

<https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.convolve.html>

- You may NOT use `np.gradient()` or other functions to compute the partial derivatives for you
- You can convolve with the following kernels to compute the partial derivatives:

- For partial derivatives in the x-direction, convolve with the kernel:

$$\begin{bmatrix} 0 & 0 & 0 \\ 0.5 & 0 & -0.5 \\ 0 & 0 & 0 \end{bmatrix}$$

- For partial derivatives in the y-direction, convolve with the kernel:

$$\begin{bmatrix} 0 & 0.5 & 0 \\ 0 & 0 & 0 \\ 0 & -0.5 & 0 \end{bmatrix}$$

- You should use `mode=same` for this problem. (What does this do?)
- You do not need to invert the height map for this problem
 - The heightmap is initialized to create a right-handed coordinate system (positive x going right, positive y going downwards).

In [16]: `heightmap.shape`

Out[16]: (188, 126)

In [20]: `display(kernx)`

```
array([[ 0. ,  0. ,  0. ],
       [ 0.5,  0. , -0.5],
       [ 0. ,  0. ,  0. ]])
```

```
In [18]: # Compute and plot the surface normals
# (make sure that the normal vector is normalized to unit length)

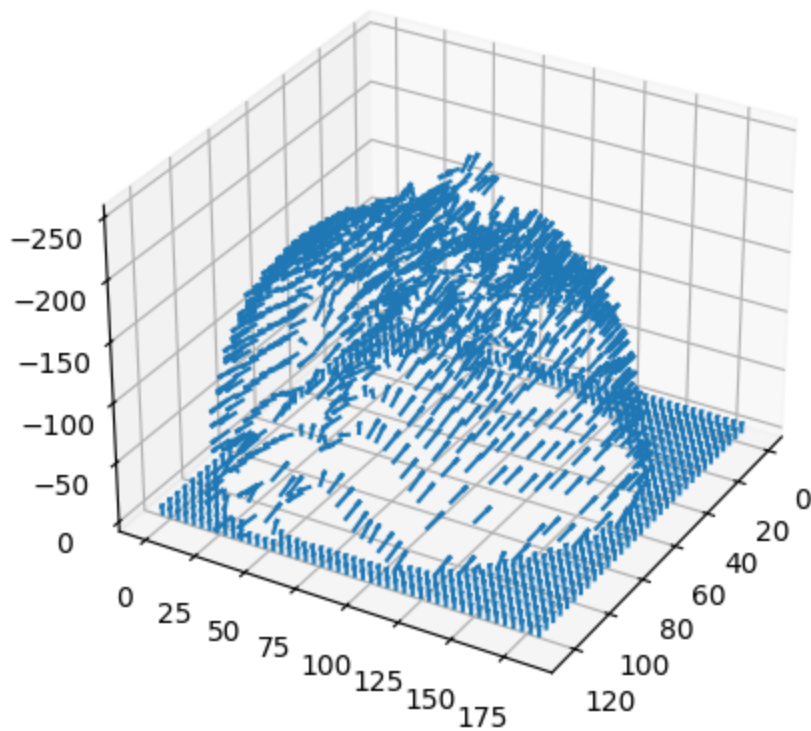
normals = np.stack((np.zeros(heightmap.shape), np.zeros(heightmap.shape), np.ones(hei

# This part of the code should compute the normals i.e fill in the above normals va
# execute the next cell to plot the surface normals

""" YOUR CODE HERE """
kernx = np.zeros((3, 3))
kerny = np.zeros((3, 3))
kernx[1, 0] = 0.5
kernx[1, 2] = -0.5
kerny[0, 1] = 0.5
kerny[2, 1] = -0.5
normals[:, :, 0] = convolve( heightmap , kernx , mode = "same")
normals[:, :, 0] = convolve( heightmap , kerny , mode = "same")
#normals = np.stack(( -dfdx, -dfdy, np.ones_like(heightmap)), axis=-1)
norm_magnitude = np.linalg.norm(normals, axis=-1, keepdims=True)
normals = normals / norm_magnitude
```

```
In [19]: # This code is used to visualise the surface normals plot.
# Feel free to change the stride variable below. However DO NOT change the other pa

stride = 4
X, Y = np.meshgrid(np.arange(0, heightmap.shape[1], stride), np.arange(0, heightmap.
Z = heightmap[::stride, ::stride]
NX = -normals[... , 0][::stride, ::stride] # Inverting to get outward normals
NY = -normals[... , 1][::stride, ::stride]
NZ = -normals[... , 2][::stride, ::stride]
fig = plt.figure(figsize=(5, 5))
ax = fig.add_subplot(projection = '3d')
xx, yy = np.mgrid[0:heightmap.shape[0], 0:heightmap.shape[1]]
ax.invert_zaxis()
ax.view_init(elev=30, azim=30)
plt.quiver(X, Y, Z, NX, NY, NZ, length=10)
plt.show()
```



2(d) Render images [8 pts]

For each of the two albedos, render three images. One for each of the two light sources, and one for both light-sources combined. Display these in a 2×3 subplot figure with titles. Recall that the general image formation equation is given by

$$I = a(x, y) \hat{\mathbf{n}}(x, y)^\top \hat{\mathbf{s}}(x, y) s_0 \quad (1)$$

where $a(x, y)$ is the albedo for pixel (x, y) , $\hat{\mathbf{n}}(x, y)$ is the corresponding surface normal, $\hat{\mathbf{s}}(x, y)$ the light source direction, s_0 the light source intensity. Let the light source intensity be 1 and make the 'distant light source assumption'. Use `imshow` with appropriate keyword arguments.

Note:

- Ensure that the images computed by `lambertian()` are within the range $[0, 1]$. Otherwise, you will see gray image background (which should be black).
 - You can consider clipping the data using `np.clip()` to be within this range.

```
In [22]: # Render Images
def lambertian(normals, light, albedo, intensity):
    """
    Given the normals, light source direction, albedo, and intensity,
    render an image according to the image formation model above.

    Parameters:
        normals: 3D array (h,w,3)
```

```

    light: 1D array (3,)          <normalize light
    albedo: 2D array (h,w)
    intensity: double

Return: 2D array (h,w)
"""

""" YOUR CODE HERE """

light = light / np.linalg.norm(light)
dprod = np.sum(normals * light, axis=-1)
image = albedo * dprod * intensity
# Clip ig
return np.clip(image, 0, 1)

```

When combining the two light sources, notice the linearity of the image formation equation. We can assume that light is additive meaning, to render an image with two light sources, we can render image 1 with light source 1 and image 2 with light source 2 independently, and then simply add images together element-wise (remember that this may cause the values outside of the range [0,1]).

$$I_{final, img} = I_{im1} + I_{im2}$$

In [23]: `display(light_source)`

```

array([[120, 180, 300],
       [ 20,  20, 300]], dtype=uint16)

```

In [24]:

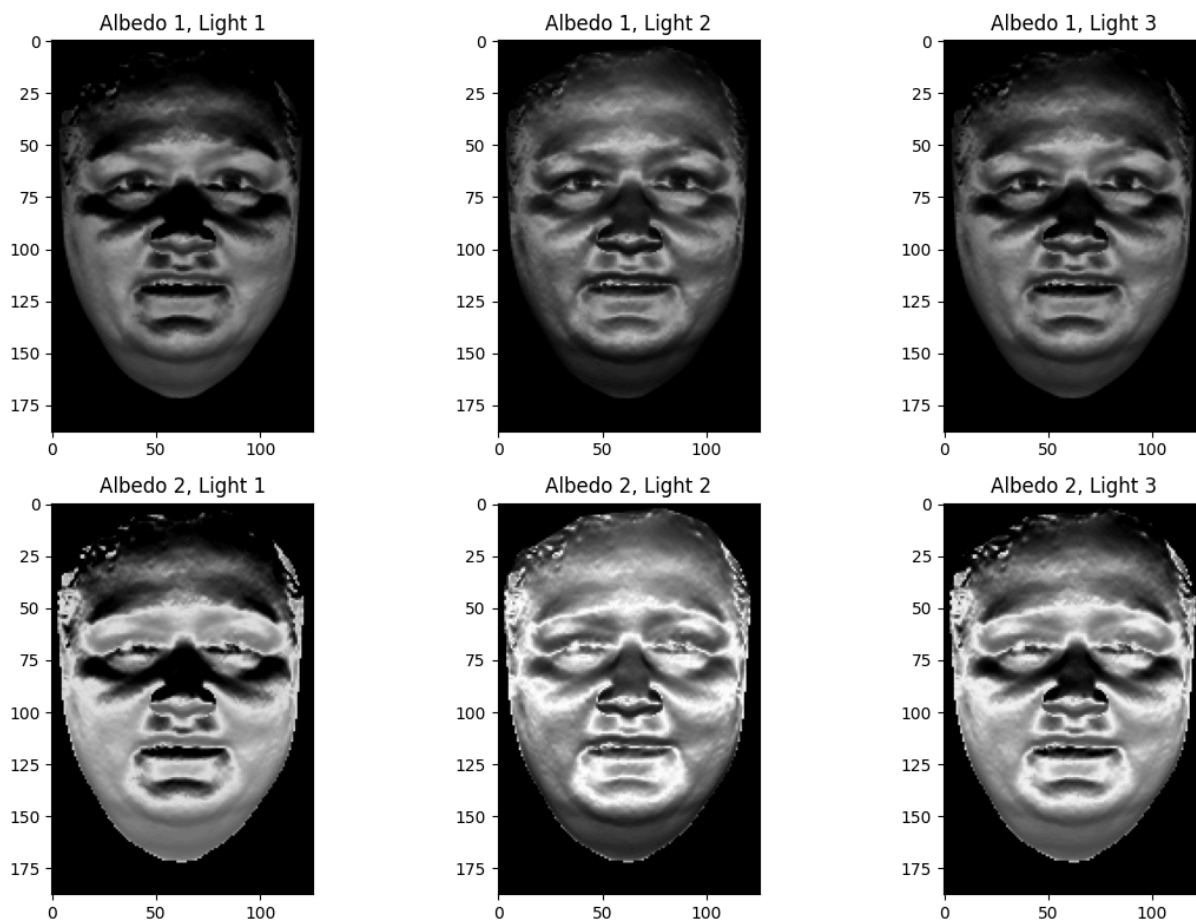
```

# For each of the two albedos, render three images.
# One for each of the two light sources, and one for both light-sources combined.
""" YOUR CODE HERE """
albedo1 = albedo
albedo2 = uniform_albedo
light1 = light_source[0]
light2 = light_source[1]
combined = light_source[0] + light_source[1]
fig, axes = plt.subplots(2, 3, figsize=(12, 8))

# Render images for each albedo and light source combination
for i, albedo in enumerate([albedo1, albedo2]):
    for j, light in enumerate([light1, light2, combined]):
        image = lambertian(normals, light, albedo, intensity=1.0) # Generate image
        axes[i, j].imshow(image, cmap='gray', vmin=0, vmax=1) # Display image
        axes[i, j].set_title(f"Albedo {i+1}, Light {j+1}")

plt.tight_layout()
plt.show()

```



Problem 3: Homogeneous Coordinates and Vanishing Points [20 points]

In class, we discussed the concept of homogeneous coordinates. In this example, we will confine ourselves to the real 2D plane. A point $(x, y)^\top$ on the real 2D plane can be represented in homogeneous coordinates by a 3-vector $(wx, wy, w)^\top$, where $w \neq 0$ is any real number. All values of $w \neq 0$ represent the same 2D point. Dividing out the third coordinate of a homogeneous point (x, y, z) converts it back to its 2D equivalent:

$$\left(\frac{x}{z}, \frac{y}{z}\right)^\top.$$

Consider a line in the 2D plane, whose equation is given by $ax + by + c = 0$. This can equivalently be written as $\mathbf{l}^\top \mathbf{x} = 0$, where $\mathbf{l} = (a, b, c)^\top$ and $\mathbf{x} = (x, y, 1)^\top$. Noticing that $\mathbf{x}$ is a homogeneous representation of $(x, y)^\top$, we define $\mathbf{l}$ as a homogeneous representation of the line $ax + by + c = 0$. Note that the line $(ka)x + (kb)y + (kc) = 0$ for $k \neq 0$ is the same as the line $ax + by + c = 0$, so the homogeneous representation of the line $ax + by + c = 0$ can be equivalently given by $(a, b, c)^\top$ or $(ka, kb, kc)^\top$ for any $k \neq 0$.

All points (x, y) that lie on the line $ax + by + c = 0$ satisfy the equation $\mathbf{l}^\top \mathbf{x} = 0$, thus, we can say that a condition for a homogeneous point $\mathbf{x}$ to lie on the homogeneous line $\mathbf{l}$ is that their dot product is zero, that is, $\mathbf{l}^\top \mathbf{x} = 0$. We note this down as a fact:

Fact 1: A point $\mathbf{x}$ in homogeneous coordinates lies on the homogeneous line $\mathbf{l}$ if and only if

$$\mathbf{x}^\top \mathbf{l} = \mathbf{l}^\top \mathbf{x} = 0$$

Now let us solve a few simple examples:

- **3(a)** Give at least two homogeneous representations for the point $(-4, 5)^\top$ on the 2D plane, one with $w > 0$ and one with $w < 0$.

[1 Point]

Answer: $(-4, 5, 1)$, $(4, -5, -1)$

- **3(b)** What is the equation of the line passing through the points $(-2, -5)^\top$ and $(-4, 3)^\top$ [in the usual Cartesian coordinates]? Now write down a 3-vector that is a homogeneous representation for this line.

[2 Points]

Answer: $(-2, -5) \rightarrow (-4, 3)$ slope = $8/-2$

$$y = -4x - 13$$

$$4x + y + 13 = 0$$

$$(4, 1, 13)^\top$$

We will now move on to consider the intersection of two lines. We make the claim that: "The (homogeneous) point of intersection, $\mathbf{x}$, of two homogeneous lines $\mathbf{l}_1$ and $\mathbf{l}_2$ is $\mathbf{x} = \mathbf{l}_1 \times \mathbf{l}_2$, where $\times$ stands for the vector (or cross) product".

- **3(c)** In plain English, how will you express the condition a point must satisfy to lie at the intersection of two lines? Armed with this simple condition, and using **Fact 1**, can you briefly explain why $\mathbf{l}_1 \times \mathbf{l}_2$ must lie at the intersection of lines $\mathbf{l}_1$ and $\mathbf{l}_2$?

[5 Points]

Answer: For a point to be on a line, the dot product of the point with the line must be zero. Knowing that this point has to be on both lines, the dot product of the point with the first line must be zero and the same for the dot product of the point with the second line. To achieve this we can cross product the two lines to find the point where this property is true so $\mathbf{l}_1 \times \mathbf{l}_2$ must lie at the intersection of lines $\mathbf{l}_1$ and $\mathbf{l}_2$

In the following, we will use the above stated claim for the intersection of two lines.

- **3(d)** Consider the two lines $4x - 3y + 2 = 0$ and $-x + y = 0$. Use the claim in question **3(c)** to find their intersection in homogeneous coordinates. Next, convert this homogeneous point back to standard Cartesian coordinates and report the 2D point of intersection.

[3 Points]

Answer: The two vectors (lines) result in the cross product of $(-2, -2, 1)$ which represents a point at $(-2, -2)$ for the 2D point of intersection

- **3(e)** Consider the two lines $-3x + 3y - 5 = 0$ and $2x - 2y + 1 = 0$. What is the special relationship between these two lines in the Euclidean plane? What is their intersection in standard Cartesian coordinates?

[2 Points]

Answer: These two lines are parallel in the Euclidean plane, we can see this by finding the slope of these two lines (-1) by dividing the rise over run as we normally would do. When taking the cross product of the lines, we get the vector $(-7, -7, 0)$. The 0 for w explains that this point doesn't exist in Cartesian coordinates

- **3(f)** Write the homogeneous representations of the above two lines from part **3(e)** and compute their point of intersection in homogeneous coordinates. What is this point of intersection called in computer vision parlance?

[3 Points]

Answer: The homogeneous representations of the lines are $(-3, 3, -5)$ and $(2, -2, 1)$ and as stated in the previous problem, the cross products of these two vectors results in the intersection of $(-7, -7, 0)$ in homogeneous coordinates. This "point" is called ideal points because they can't be converted to cartesian points.

- **3(g)** Do questions **3(e)** and **3(f)** justify the claim in class, that homogeneous coordinates provide a uniform treatment of line intersection, regardless of parallelism? Briefly explain.

[2 Points]

Answer: The homogeneous coordinates provide uniform treatment because they will always compute or result in a point no matter the two lines passed in. Without the homogenous coordinates, two parallel lines would not result in an answer, however with homogenous coordinates even parallel lines result in an intersection point as exemplified in the answer above.

- **3(h)** Give (with justification) an expression for the homogeneous representation of the line passing through two homogeneous points $\mathbf{x}_1$ and $\mathbf{x}_2$. [Hint: Construct an argument analogous to the one for the intersection of two lines in part **3(c)**.]

[2 Points]

Answer: The idea here is essentially the same as in 3(c), in order to find a point that lies on a line, the result of the dot product of that point and the line must be zero. In this case there are two points and we simply need to make sure they are both on the same line. Therefore,

the line in which the dot product of the line and the first point and dot product of the line and the second point are both zero is the cross product of the two points. The cross product only works in 3 dimensions so it will work with the homogenous points:

$$\mathbf{x}_1 \times \mathbf{x}_2 = \mathbf{l}_i$$

Problem 4: Camera Matrices and Rigid-Body Transformations [20 points]

Consider a world coordinate system W , centered at the origin $(0, 0, 0)$, with axes given by unit vectors $\hat{\mathbf{i}} = (1, 0, 0)^\top$, $\hat{\mathbf{j}} = (0, 1, 0)^\top$ and $\hat{\mathbf{k}} = (0, 0, 1)^\top$. We use a notation where boldfaces stand for a vector and a hat above a boldface letter stands for a unit vector.

- **4(a)** Consider another coordinate system, with unit vectors along two of the orthogonal axes as:

$\hat{\mathbf{i}}' = (0.688, -0.688, 0.229)^\top$ and $\hat{\mathbf{j}}' = (0.267, 0.534, 0.801)^\top$. Find the unit vector, $\hat{\mathbf{k}}'$, along the third axis orthogonal to both $\hat{\mathbf{i}}'$ and $\hat{\mathbf{j}}'$. Is there a unique such unit vector? If not, choose the one that makes an acute angle with $\hat{\mathbf{k}}$.

[2 Points]

Answer: to find $\hat{\mathbf{k}}'$ we would just take the cross product of the two other vectors because $\hat{\mathbf{k}}'$ should be orthogonal to the other two vectors. To make an acute angle with the original $\hat{\mathbf{k}}$ we just need to make sure the dot product of the new $\hat{\mathbf{k}}'$ with the original is positive, which means we choose whichever orthogonal vector has a positive z component. Plugging in the cross product:

$$\hat{\mathbf{k}}' = [-0.673, -0.490, 0.551]$$

- **4(b)** Find the rotation matrix that rotates any vector in the $(\hat{\mathbf{i}}, \hat{\mathbf{j}}, \hat{\mathbf{k}})$ coordinate system to the $(\hat{\mathbf{i}}', \hat{\mathbf{j}}', \hat{\mathbf{k}}')$ coordinate system.

[2 Points]

!! Hint !!

The rotation matrix is given by $R =$

$$\begin{bmatrix} u^T \\ v^T \\ w^T \end{bmatrix}$$

Answer: Since the three vectors for each coordinate system form a basis for their entire coordinate system, we can simply put the unit vectors we found into the rotational matrix (as the hint suggests).

$$\begin{bmatrix} 0.688 & -0.688 & 0.229 \\ 0.267 & 0.534 & 0.801 \end{bmatrix}$$

- **4(c)** What is the *extrinsic parameter matrix* for a camera at a displacement $(-1, -2, -5)^\top$ from the origin of W and oriented such that its principal axis coincides with $\hat{\mathbf{k}}'$, the x-axis of its image plane coincides with $\hat{\mathbf{i}}'$ and the y-axis of the image plane coincides with $\hat{\mathbf{j}}'$?

[3 Points]**!! Hint !!**

The extrinsic parameter matrix is given by:

$$\begin{pmatrix} R & -RC \\ 0^T & 1 \end{pmatrix}$$

where R is the rotation matrix and C is the camera center.

Answer: given that $-RC =$

$$\begin{bmatrix} 0.457 \\ 5.34 \\ 1.102 \end{bmatrix}$$

we can simply concatenate based on the format given above:

$$\begin{bmatrix} 0.688 & -0.688 & 0.229 & 0.457 \\ 0.267 & 0.534 & 0.801 & 5.34 \\ -0.673 & -0.490 & 0.551 & 1.102 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

(matrix calculators had a hard time with this for some reason)

- **4(d)** What is the *intrinsic parameter matrix* for this camera, if its focal length in the x-direction is 1024 pixels, aspect ratio is 1, pixels deviate from rectangular by 0 degrees and principal point is offset from the center $(0, 0)^\top$ of the image plane to the location $(10, -5)^\top$?

[3 Points]

Answer:

$$\begin{bmatrix} -1024 & 0 & 10 \\ 0 & -1024 & -5 \\ 0 & 0 & 1 \end{bmatrix}$$

- **4(e)** Write down the projection matrix for the camera described by the configuration in parts **4(c)** and **4(d)**.

[3 Points]**!! Hint !!**

The projection matrix is given by $P = K [R \mid -RC]$ where K is the intrinsic matrix, R is the rotation matrix and C is the camera center.

Answer: $P =$

$$\begin{bmatrix} 1024 & 0 & 10 \\ 0 & 1024 & -5 \\ 0 & 0 & 1 \end{bmatrix}$$

* (multiply)

$$\begin{bmatrix} 0.688 & -0.688 & 0.229 & 0.457 \\ 0.267 & 0.534 & 0.801 & 5.34 \\ -0.673 & -0.490 & 0.551 & 1.102 \end{bmatrix}$$

= (equal to) (P is shown below)

$$\begin{bmatrix} 697.782 & -709.412 & 240.0 & 456.948 \\ 276.773 & 544.366 & 817.979 & 5473.67 \\ -0.673 & -0.490 & 0.551 & 1.102 \end{bmatrix}$$

- **4(f)** Consider a plane, orthogonal to $\hat{\mathbf{k}}$, at a displacement of 2 units from the origin of W along the $\hat{\mathbf{k}}$ direction. Consider a disk with radius 1, centered at $(0, 0, 2)^\top$ in the coordinate system W . We wish to find the image of this disk, as seen by the camera we constructed in part **4(e)**. The following questions need programming (use Python) and the code for each part should be turned in along with any figures and answers to specific questions. Explain your variable names (with comments). Feel free to supply any additional description or explanation to go with your code.

1. Compute 10000 well-distributed points on the surface of unit disk with a hole inside (a circular annulus). The inner radius of the hole is $r_{inner} = 0.5$, and the outer radius is $r_{outer} = 1$. One way to do this is to sample the angular range 0 to 360 degrees into 10000 equal parts and convert the resulting points from polar coordinates to Cartesian coordinates. Display the resulting annulus, make sure that the axes of the display figure are equal.

[2 Points]

!! Hint !!

The following code snippet generates one such random point:

```
r = sqrt(R_inner^2 + random() * (R_outer^2 - inner^2))
theta = random() * 2 * PI`
```

2. Add the z coordinate to these points, which is 2 for all of them. Make all the points homogeneous by adding a fourth coordinate equal to 1.

[1 Point]

3. Compute the projection of these homogeneous points using the camera matrix from part **4(e)**. Convert the homogeneous projected points to 2D Cartesian points by dividing out (and subsequently discarding) the third coordinate of each point.

[2 Points]

4. Plot the projected 2D points, again ensure that the axes of your plot are equal. What is the shape of the image of a disk?

[2 Points]

```
In [29]: R_inner = 0.5
R_outer = 1.0
num_points = 10000

# Random radius and angle
r = np.sqrt(R_inner**2 + np.random.rand(num_points) * (R_outer**2 - R_inner**2))
theta = np.random.rand(num_points) * 2 * np.pi

#pole to cart
x = r * np.cos(theta)
y = r * np.sin(theta)

# cart to hom
z = np.full_like(x, 2)
homogeneous_points = np.vstack((x, y, z, np.ones_like(x))) # Add coord

#project matrix
camera_matrix = np.array([
    [697.782, -709.412, 240.0, 479.01],
    [276.773, 544.366, 822.979, 5463.4],
    [-.673, -.49, .551, 1.102]
])

# Project the points
projected_points = camera_matrix @ homogeneous_points
projected_points /= projected_points[2, :] #conv back to cart

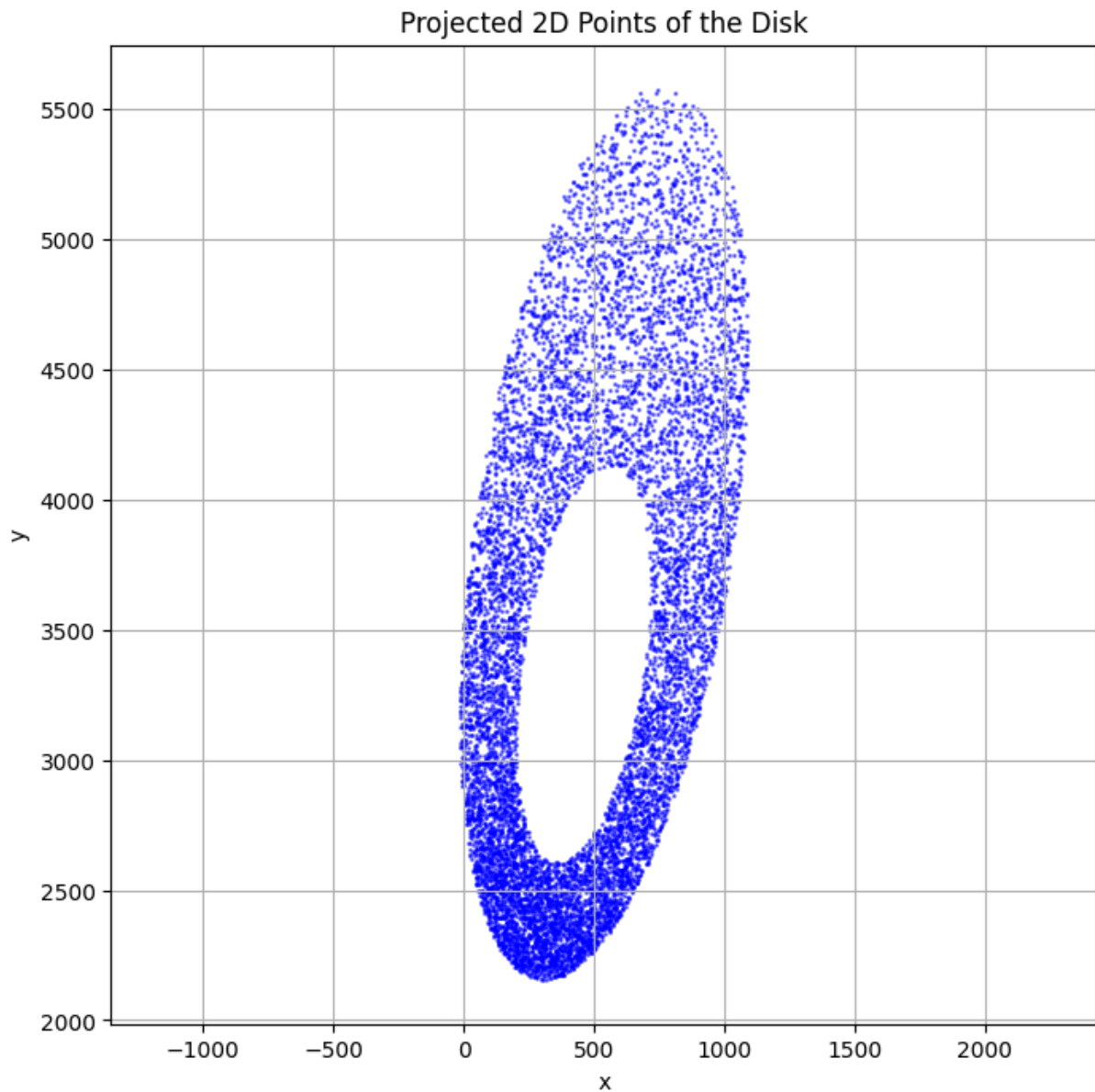
# into 2D
x_proj = projected_points[0, :]
```

```

y_proj = projected_points[1, :]

plt.figure(figsize=(8, 8))
plt.scatter(x_proj, y_proj, s=1, color='blue', alpha=0.5)
plt.title("Projected 2D Points of the Disk")
plt.xlabel("x")
plt.ylabel("y")
plt.axis('equal') # Ensure equal scaling for both axes
plt.grid(True)
plt.show()

```



```

In [28]: # same code but now looking from above (camera now fixed into a view directly on th
R_inner = 0.5
R_outer = 1.0
num_points = 10000

# Random radius and angle
r = np.sqrt(R_inner**2 + np.random.rand(num_points) * (R_outer**2 - R_inner**2))
theta = np.random.rand(num_points) * 2 * np.pi

```

```
#pole to cart
x = r * np.cos(theta)
y = r * np.sin(theta)

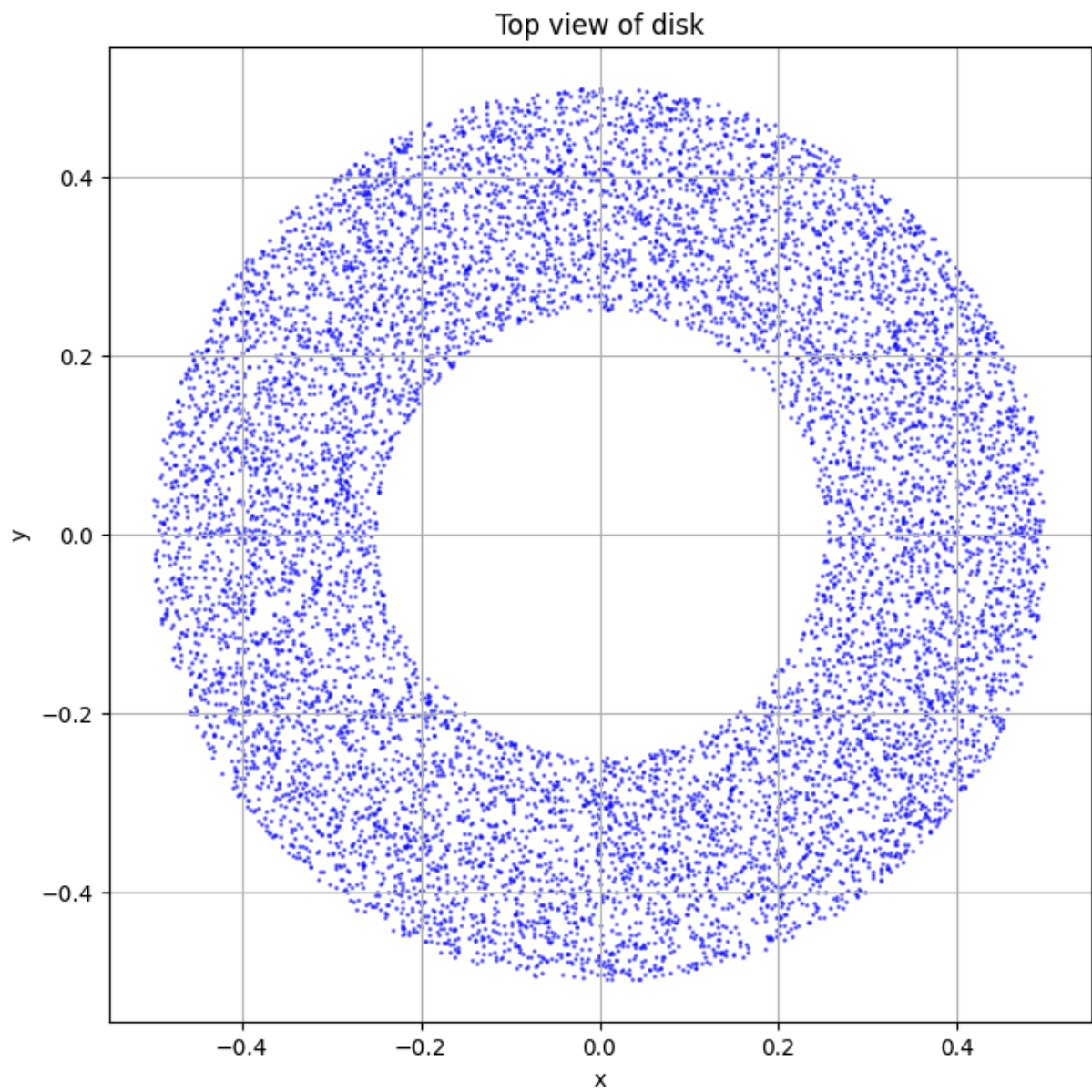
# cart to hom
z = np.full_like(x, 2)
homogeneous_points = np.vstack((x, y, z, np.ones_like(x))) # Add coord

#project matrix
camera_matrix = np.array([
    [1, 0, 0, 0],
    [0, 1, 0, 0],
    [0, 0, 1, 0]
])

# Project the points
projected_points = camera_matrix @ homogeneous_points
projected_points /= projected_points[2, :] #conv back to cart

# into 2D
x_proj = projected_points[0, :]
y_proj = projected_points[1, :]

plt.figure(figsize=(8, 8))
plt.scatter(x_proj, y_proj, s=1, color='blue', alpha=0.5)
plt.title("Top view of disk")
plt.xlabel("x")
plt.ylabel("y")
plt.axis('equal') # Ensure equal scaling for both axes
plt.grid(True)
plt.show()
```



In []: